# NPM and Package Management

**Section:** 2 of 11 | **Subtopic:** NPM Fundamentals | **Duration:** 4 – 5 hours total | Theory | Practical

**Topics covered:** What is NPM · The Registry · `npm init` · `package.json` anatomy · Installing packages · Custom scripts

### Learning Objectives — This Subtopic

- Understand what NPM is and the problem it solves.
- Initialise a Node.js project with `npm init` and understand every field in the generated `package.json`.
- Install packages locally and understand what `node_modules` and `package-lock.json` are.
- Distinguish between `dependencies` and `devDependencies`.
- Write and run custom NPM scripts.

## 1. What is NPM?

NPM stands for **Node Package Manager**. It is two things at once:

1. A **command-line tool** (the `npm` command) that ships with every Node.js installation. You use it to install, remove, and manage code packages.
2. An **online registry** at npmjs.com — a massive public database of over 2 million open-source JavaScript packages that anyone can publish or consume for free.

### Analogy

> Think of NPM like a smartphone app store. The *registry* is the store's catalogue — millions of apps (packages) written by other developers. The *npm CLI* is the "Install" button. Instead of manually copying code from the internet into your project, you run one command and NPM downloads the exact package you need, at the exact version you want, along with everything that package itself depends on.

Before package managers existed, developers had to manually download libraries, place them in the correct folder, and keep track of versions themselves. NPM automates all of this.

### What is a "Package"?

A package is simply a folder containing JavaScript code along with a `package.json` file that describes it. That code might be a utility library (e.g., `lodash` for array manipulation), a framework (e.g., `express`), a testing tool (e.g., `jest`), or anything else.

> **Note:** When people say "installing a library" or "installing a dependency", they mean the same thing as "installing a package".

---

## 2. Initialising a Project: `npm init`

Every Node.js project should have a `package.json` file at its root. This file is the identity card of your project — it records the project name, version, entry point, scripts, and most importantly, the list of all packages your project depends on.

You create this file by running `npm init` inside your project folder.

### Step-by-step: Starting a new project

```
# 1. Create a new folder for your project
mkdir my-first-app
cd my-first-app
```

```
# 2. Initialise NPM — this launches an interactive questionnaire
npm init
```

NPM will ask you a series of questions. You can answer each one or press `Enter` to accept the default value shown in parentheses.

**Output:**

```
This utility will walk you through creating a package.json file.

package name: (my-first-app)      ← press Enter to accept default
version: (1.0.0)                  ← press Enter
description: My first Node.js app ← type a description, then Enter
entry point: (index.js)           ← press Enter
test command:                     ← press Enter (leave blank for now)
git repository:                   ← press Enter
keywords:                         ← press Enter
author: Alice                     ← type your name
license: (ISC)                    ← press Enter

Is this OK? (yes) yes
```

After confirming, NPM writes a `package.json` file to your folder. There is also a faster way to skip all questions and accept every default:

```
npm init -y
```

The `-y` flag means "yes to everything". Use this when you want to get started quickly. You can always edit the generated file manually afterwards.

---

## 3. Anatomy of `package.json`

After running `npm init -y`, you will have a file that looks like this:

`package.json`

```
{
  "name": "my-first-app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}
```

Every field has a specific purpose. The table below explains each one:

| Field | Purpose | Required? |
|---|---|---|
| `"name"` | The unique name of your project or package. If you publish to the NPM registry, this must be globally unique. Use lowercase letters and hyphens only. | Yes (if publishing) |
| `"version"` | The current version of your project using semantic versioning (covered in the next subtopic). Always starts at `"1.0.0"`. | Yes (if publishing) |
| `"description"` | A short human-readable description of what the project does. Used in the NPM registry search results. | No |
| `"main"` | | No (defaults to index.js) |

| Field | Purpose | Required? |
| --- | --- | --- |
| | The entry point file of your application. When someone imports your package, this is the file that runs first. Defaults to `index.js`. | |
| `"scripts"` | An object of custom command shortcuts. Run them with `npm run <name>`. Covered in detail in Section 3 of this topic. | No |
| `"keywords"` | An array of strings that help users find your package on the NPM registry via search. | No |
| `"author"` | Your name or organisation. Can also be an object with `name`, `email`, and `url`. | No |
| `"license"` | Tells others how they are allowed to use your code. Common values: `"MIT"`, `"ISC"`, `"Apache-2.0"`. | No |
| `"dependencies"` | Packages that your application needs to *run* in production. Created automatically when you install a package. (Covered below.) | Auto-generated |
| `"devDependencies"` | Packages only needed during *development* (e.g., testing tools, linters). Not required in production. (Covered below.) | Auto-generated |

**Key Insight:** `package.json` is the single source of truth for your project's configuration. Whenever another developer clones your project, they run `npm install` and NPM reads this file to know exactly which packages to download. This is how teams share projects without committing hundreds of megabytes of dependencies into version control.

---

## 4. Installing Packages

The command to install a package is:

```
npm install <package-name>

# Short form — identical result:
npm i <package-name>
```

Let us install a real package. `chalk` is a popular library for adding colour to terminal output. It is a good first package to learn with because its effect is immediately visible.

```
npm install chalk
```

**Output:**

```
added 1 package, and audited 2 packages in 1s

1 package is looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Three things just happened in your project folder:

```
my-first-app/ ├── node_modules/ ← NEW: downloaded package files live here │ └── chalk/ │ ├── index.js │ └── package.json ├── package-lock.json ← NEW: exact version lock file (explained below) └── package.json ← UPDATED: chalk added to "dependencies"
```

### 4.1 What changed in `package.json`?

Open `package.json` — you will now see a new `"dependencies"` section:

`package.json (after install)`

```
{
  "name": "my-first-app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "chalk": "^5.3.0"    ← NPM recorded the installed package and its version
  }
}
```

NPM automatically added `chalk` to `"dependencies"` and recorded the version it installed. The `^` before the version number is a version range specifier — covered in the next subtopic.

### 4.2 What is `node_modules`?

The `node_modules` folder is where NPM physically downloads and stores the code for every installed package. You should never manually edit files inside this folder.

> **Analogy**

> Think of `node_modules` as a supplier warehouse attached to your project. `package.json` is your purchase order list. NPM reads the list, fetches the goods from the online registry, and places them in the warehouse. If the warehouse burns down (you delete `node_modules`), you can rebuild it entirely just by re-running `npm install` — because the purchase order (`package.json`) still exists.

> **Important — Always add `node_modules` to `.gitignore`:**
> The `node_modules` folder can easily contain tens of thousands of files and hundreds of megabytes. You should *never* commit it to version control. Instead, commit only `package.json` and `package-lock.json`. Any developer who clones your project can restore `node_modules` with a single `npm install` command.
>
> Create a `.gitignore` file in your project root and add:
>
> ```
> node_modules/
> ```

## 4.3 What is `package-lock.json`?

`package-lock.json` is automatically generated and updated by NPM. It records the *exact* version of every package (and every package's dependency) that was installed at the time of installation.

Consider this distinction:

- `package.json` says: "I need chalk version 5.x.x or higher" (approximate)
- `package-lock.json` says: "chalk version 5.3.0 specifically was installed" (exact)

This matters because if a new version of a package is released between when you develop and when a colleague installs the project, without a lock file they would get a different version. With `package-lock.json`, `npm install` installs the exact same versions for everyone on the team, eliminating the classic "it works on my machine" problem.

> **Rule of thumb:** Always commit `package-lock.json` to version control. Never commit `node_modules`.

## 4.4 `dependencies` vs `devDependencies`

Not every package you install is needed in a live production environment. Testing frameworks, linters, and build tools are only needed while you are developing. NPM separates these two categories:

| Type | Field in package.json | Install command | When is it needed? | Example packages |
|---|---|---|---|---|
| Production dependency | `"dependencies"` | `npm install <pkg>` | Development *and* production | `express`, `chalk`, `axios` |
| Development dependency | `"devDependencies"` | `npm install <pkg> --save-dev` or `-D` | Development only — not needed to run the app | `jest`, `nodemon`, `eslint` |

Let us install `nodemon` as a dev dependency. Nodemon is a tool that automatically restarts your Node.js application whenever you save a file — very useful during development, but not needed when the app is deployed.

```
npm install nodemon --save-dev
```

Your `package.json` now has both sections:

`package.json`

```
{
  "name": "my-first-app",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "chalk": "^5.3.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"     ← saved here because of --save-dev
  }
}
```

When deploying to a production server, you can install *only* production dependencies (skip devDependencies) with:

```
npm install --omit=dev

# Older syntax (still widely used):
npm install --production
```

This keeps the production environment lean and avoids shipping development tools to a live server.

---

## 5. Using an Installed Package in Code

Once a package is installed, you bring it into your JavaScript file using `require()` (CommonJS — the default in Node.js). Modules are covered in full in Section 4, but you need to know the basics here to use any package.

### Example 1 — Installing and Using a Package ( `chalk` )

We already installed `chalk`. Now create `index.js` and use it:

`index.js`

```
// require() loads an installed package from node_modules.
// The string inside must exactly match the package name.
const chalk = require('chalk');

// chalk provides methods to colour terminal text.
console.log(chalk.green('Success: Server started!'));
console.log(chalk.red('Error: Something went wrong.'));
console.log(chalk.blue.bold('Info: Listening on port 3000'));
console.log(chalk.yellow('Warning: Deprecated function used.'));
```

```
node index.js
```

Output:

```
Success: Server started!
Error: Something went wrong.
Info: Listening on port 3000
Warning: Deprecated function used.
```

**What is happening?** `require('chalk')` tells Node.js to look inside `node_modules/chalk/` and load its exported code. The `chalk` object returned has methods like `.green()`, `.red()`, etc. — all of which wrap your text in ANSI colour codes that the terminal renders as colour.

Notice you did not write any colour logic yourself. This is exactly the value of packages — you use tested, maintained code that others have already written.

## Example 2 — Restoring `node_modules` from `package.json`

This example demonstrates one of the most important concepts in NPM: the entire `node_modules` folder is disposable and reproducible.

Delete the `node_modules` folder entirely:

```
# macOS / Linux
rm -rf node_modules

# Windows (Command Prompt)
rmdir /s /q node_modules
```

Now try running the script — it will fail because the packages are gone:

```
node index.js
```

Output:

```
node:internal/modules/cjs/loader:1092
  throw err;

Error: Cannot find module 'chalk'
```

Now restore everything with a single command:

```
npm install
```

> **Output:**
>
> ```
> added 2 packages, and audited 3 packages in 0.8s
>
> found 0 vulnerabilities
> ```

Run the script again — it works perfectly. NPM read `package.json`, saw `chalk` and `nodemon` listed, and downloaded them again. This is the entire purpose of recording dependencies in `package.json`.

---

## 6. NPM Scripts

The `"scripts"` field in `package.json` lets you define custom command shortcuts. Instead of remembering long, complex terminal commands, you give them simple names and run them with `npm run <name>`.

> **Analogy**
>
> Think of NPM scripts as keyboard shortcuts on your phone. Instead of navigating to Settings → Display → Brightness every time, you create a shortcut. Similarly, instead of typing a long command every time, you create a script with a memorable name and run it with two words.

### 6.1 Built-in Script Names vs Custom Names

NPM treats three script names specially — they can be run *without* the `run` keyword:

| Script name | Run with | Typical use |
|---|---|---|
| `start` | `npm start` | Start the application (production) |

| Script name | Run with | Typical use |
|---|---|---|
| `test` | `npm test` | Run the test suite |
| `stop` | `npm stop` | Stop the application |

Any other name you invent must be run with `npm run <name>`. For example: `npm run dev`, `npm run build`, `npm run lint`.

## Example 3 — Writing and Running Scripts Progressively

We will build up the `"scripts"` section in three stages, each time adding a more useful script.

### Stage 1 A basic start script

Update the `"scripts"` section in `package.json`:

`package.json — Stage 1`

```
{
  "name": "my-first-app",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test":  "echo \"No tests yet\""
  },
  "dependencies": {
    "chalk": "^5.3.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

```
npm start
```

> **Output:**

```
> my-first-app@1.0.0 start
> node index.js

Success: Server started!
Error: Something went wrong.
Info: Listening on port 3000
Warning: Deprecated function used.
```

NPM prints the script name and the command it is running before executing it. This is normal behaviour.

### Stage 2 Adding a `dev` script with nodemon

During development, you want the server to restart automatically every time you save a file. `nodemon` does this. Add a `"dev"` script:

`package.json — Stage 2`

```
{
  "name": "my-first-app",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev":   "nodemon index.js",
    "test":  "echo \"No tests yet\""
  },
  "dependencies": {
    "chalk": "^5.3.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

```
npm run dev
```

**Output:**

```
> my-first-app@1.0.0 dev
> nodemon index.js
```

```
[nodemon] 3.1.0
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
Success: Server started!
...
[nodemon] clean exit - waiting for changes before restart
```

Now, whenever you edit and save `index.js`, nodemon automatically re-runs it. Press `Ctrl`+`C` to stop it.

> **Why `nodemon` goes in `devDependencies`:** In production, the code does not change. A live server does not need nodemon watching for file changes. Keeping it in `devDependencies` means it will not be installed when you deploy, keeping the production environment clean and fast.

### Stage 3 Passing environment variables and chaining commands

Scripts can also set environment variables, pass flags, or chain multiple commands together. Update the scripts to include an `info` script and an environment-aware `start` command:

`package.json — Stage 3`

```
{
  "name": "my-first-app",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start":      "NODE_ENV=production node index.js",
    "dev":        "NODE_ENV=development nodemon index.js",
    "info":       "node --version && npm --version",
    "clean":      "echo Cleaning build artifacts... && echo Done.",
    "test":       "echo \"No tests yet\""
  },
  "dependencies": {
    "chalk": "^5.3.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

```
npm run info
```

**Output:**

```
> my-first-app@1.0.0 info
> node --version && npm --version

v20.11.0
10.2.4
```

**What is happening?**

- `NODE_ENV=production` before a command sets that environment variable for the duration of that single command. Your code reads it with `process.env.NODE_ENV`.
- `&&` chains two commands: the second only runs if the first succeeds (exits with code 0).
- NPM scripts have access to all locally installed binaries (like `nodemon`) without you needing to type the full path to `node_modules/.bin/nodemon`. NPM handles this automatically.

## 6.2 Accessing the `npm_package_*` Variables in Scripts

Inside any NPM script, every field in your `package.json` is available as an environment variable with the prefix `npm_package_`. This is useful for logging the project name and version without hardcoding them.

`package.json`

```
{
  "name": "my-first-app",
  "version": "1.0.0",
  "scripts": {
    "version-check": "echo Running $npm_package_name version $npm_package_version"
  }
}
```

```
npm run version-check
```

**Output:**

```
Running my-first-app version 1.0.0
```

**Windows note:** The `$npm_package_name` syntax works on macOS/Linux. On Windows Command Prompt, use `%npm_package_name%`. To write cross-platform scripts, consider using the `cross-env` package, which is covered in the next subtopic.

---

## Summary

| Concept | Key Point |
|---|---|
| NPM | A CLI tool and online registry for managing JavaScript packages. Installed automatically with Node.js. |
| `npm init` | Creates `package.json`. Use `-y` flag to skip the questionnaire. |
| `package.json` | The identity and dependency manifest of your project. The most important file in any Node.js project. |
| `npm install <pkg>` | Downloads a package into `node_modules` and records it in `"dependencies"`. |
| `--save-dev` / `-D` | Installs a package as a development-only dependency into `"devDependencies"`. |
| `node_modules` | Folder where package code lives. Never commit to version control. Recreate with `npm install`. |
| `package-lock.json` | Locks exact installed versions for reproducibility. Always commit this file. |

| Concept | Key Point |
| --- | --- |
| NPM Scripts | Custom command shortcuts in `"scripts"`. Run with `npm run <name>`. `start`, `test`, `stop` are special — no `run` needed. |

---

## Interview Questions

1. **What is the difference between `dependencies` and `devDependencies` in `package.json`?**
   *Expected answer:* `dependencies` are packages required for the application to run in production (e.g., `express`). `devDependencies` are packages only needed during development (e.g., `jest`, `nodemon`). When deploying, you can skip dev dependencies with `npm install --omit=dev` to keep the server lean.

2. **What is `package-lock.json` and why should you commit it to version control?**
   *Expected answer:* It records the exact version of every installed package (and their sub-dependencies). It ensures that `npm install` produces the identical dependency tree on every machine — eliminating version inconsistencies between team members and CI/CD environments.

3. **Why should `node_modules` never be committed to Git?**
   *Expected answer:* It can contain hundreds of megabytes and tens of thousands of files. It is fully reproducible from `package.json` + `package-lock.json` by running `npm install`. Committing it wastes storage and causes unnecessary merge conflicts.

4. **What happens when you run `npm install` in a project that already has a `package.json` but no `node_modules`?**
   *Expected answer:* NPM reads `package.json` (and `package-lock.json` if present) and downloads all listed packages from the registry into a new `node_modules` folder, restoring the project to a working state.

5. **How do you run a custom script called `"build"` defined in `package.json`?**
   *Expected answer:* `npm run build`. The `run` keyword is required for custom (non-built-in) script names. Built-in names like `start` and `test` can be run without `run`.

6. **What is the purpose of the `"main"` field in `package.json`?**
   *Expected answer:* It specifies the entry point file of the package — the file that gets loaded when another project does `require('your-package-name')`. It defaults to `index.js` if not specified.

7. **Can you have multiple scripts that call other scripts inside `package.json`?**
   *Expected answer:* Yes. You can use `npm run <other-script>` inside a script's command string. For example: `"build-and-start": "npm run build && npm start"`. This is a common pattern for chaining pipeline steps.

## Practical Assignment

**Title:** Project Setup and Dependency Workflow

**Objective:** Practice the complete NPM project setup workflow end-to-end.

**Instructions:**

1. Create a new folder called `npm-practice` and initialise it as an NPM project using `npm init -y`.
2. Install the following packages appropriately:
   - `chalk` — as a production dependency
   - `nodemon` — as a dev dependency
   - `dayjs` — as a production dependency (a lightweight date utility library)
3. Create an `index.js` file that:
   - Uses `chalk` to print the project name in **bold blue**
   - Uses `dayjs` to print today's date in the format `DD/MM/YYYY`
   - Reads and prints `process.env.NODE_ENV` (or `"not set"` if undefined)

4. Add the following scripts to `package.json`:
    - `"start"` — runs `index.js` with `NODE_ENV=production`
    - `"dev"` — runs `index.js` with nodemon and `NODE_ENV=development`
    - `"info"` — prints Node.js version and NPM version
5. Delete `node_modules` and verify you can restore the project fully with `npm install`.
6. Run both `npm start` and `npm run dev` and confirm they work correctly.

**Expected output when running `npm start`:**

> Output:
>
> ```
> npm-practice
> Today: 16/02/2026
> Environment: production
> ```

**Bonus challenge:** Add a `"reset"` script that deletes `node_modules` and re-installs all packages in one command. Hint: use `rm -rf node_modules && npm install`.

---